
```

function [f,J] = Burgers1D_DF_conservativo_Residuo1(xv, epsilon, uv, CalcJ)

%Distancia entre nodos (OJO! Se supone mallado uniforme):
Nx      = length(xv);
hx      = xv(2)-xv(1);

%Reservamos memoria para residuo y Jacobiano:
f       = zeros(Nx,1);
if CalcJ
    filas = [];
    cols  = [];
    vals  = [];
end

%Puntos interiores:
for ii=2:Nx-1

    %Imponemos
    % (1/2*u_(i+1)^2 - 1/2*u_(i-1)^2)/(2*hx) - ...
    % epsilon*( u_(i+1)-2*u_(i)+u_(i-1) )/hx^2 = 0
    f(ii) = (0.5*uv(ii+1)^2 - 0.5*uv(ii-1)^2)/(2*hx) - ...
            epsilon*( uv(ii+1)-2*uv(ii)+uv(ii-1) )/hx^2;
    if CalcJ

        %Derivadas de f(I) respecto de...:
        %Nodo central (ii):
        dfI_du_C = 2*epsilon/hx^2;
        %Nodo izquierdo (ii-1)
        dfI_du_L = -uv(ii-1)/(2*hx) - epsilon/hx^2;
        %Nodo derecho (ii+1):
        dfI_du_R = +uv(ii+1)/(2*hx) - epsilon/hx^2;

        %Almacenamos derivadas:
        filas = cat(1, filas, ii, ii, ii);
        cols  = cat(1, cols, ii, ii-1, ii+1);
        vals  = cat(1, vals, dfI_du_C, dfI_du_L, dfI_du_R);

    end

end

%Frontera izquierda. Imponemos u_1 = 1:
f(1) = uv(1) - 1.0;
if CalcJ
    filas = cat(1, filas, 1);
    cols  = cat(1, cols, 1);
    vals  = cat(1, vals, 1);
end

%Frontera derecha. Imponemos u_Nx = 0:
f(Nx) = uv(Nx) - 0.0;
if CalcJ

```

```
        filas    = cat(1, filas, Nx);
        cols     = cat(1, cols, Nx);
        vals     = cat(1, vals, 1);
    end

    %Construimos jacobiano:
    if CalcJ
        J        = sparse(filas, cols, vals, Nx, Nx);
    else
        J        = NaN;
    end
end
```

Published with MATLAB® R2025a

```

function Burgers1D_DF(epsilon, Nx)

    %Creamos mallado:
    xv = linspace(0.0, 2.0, Nx).';

    %    %Probar esto con Nx, Ny pequeños y luego comentar:
    %    %Probamos funci'on:
    %    uv0 = rand(Nx,1);
    %    [f0, J0] = Burgers1D_DF_Residuo1(xv, epsilon, uv0, true);
    %    Jnum = JacobianEst(@(u) Burgers1D_DF_Residuo1(xv, epsilon, u,
false), uv0, 1e-6);
    %    display(norm(Jnum-J0,Inf))
    %    return

    %Aproximaci'on inicial:
    uv0 = 1.0-0.5*xv;

    %Resolvemos con Newton:
    tic
    fun = @(u, CalcJ) Burgers1D_DF_Residuo1(xv, epsilon, u,
CalcJ);
    [uv, nIters, flag] = NewtonRaphson1(fun, uv0, 1e-8, 200);
    if flag<0
        warning('Newton no ha convergido')
    end
    toc
    display(nIters)

    %Soluci'on exacta en los nodos de la malla u=tanh((2-x)/(2*epsilon)) y
error cometido:
    uv_ex = tanh((2-xv)/(2*epsilon));

    %Representamos soluci'on:
    figure()
    plot(xv, uv_ex, 'xb')
    hold on
    plot(xv, uv, 'r')
    xlabel('x')
    ylabel('u')
    legend('ex.', 'num.')

    %Mostramos error y tama'no de malla:
    err = norm(uv-uv_ex, Inf);
    display(err)
    hx = xv(2)-xv(1);
    display(hx)

end

```

```

function [f,J] = Burgers1D_DF_Residuo1(xv, epsilon, uv, CalcJ)

%Distancia entre nodos (OJO! Se supone mallado uniforme):
Nx      = length(xv);
hx      = xv(2)-xv(1);

%Reservamos memoria para residuo y Jacobiano:
f       = zeros(Nx,1);
if CalcJ
    filas = [];
    cols  = [];
    vals  = [];
end

%Puntos interiores:
for ii=2:Nx-1

    %Imponemos
    %  $u_i(u_{i+1}-u_{i-1})/(2*hx) - \dots$ 
    %  $\epsilon(u_{i+1}-2*u_i+u_{i-1})/hx^2 = 0$ 
    f(ii) = uv(ii) * (uv(ii+1) - uv(ii-1))/(2*hx) - ...
            epsilon*( uv(ii+1)-2*uv(ii)+uv(ii-1) )/hx^2;
    if CalcJ

        %Derivadas de f(I) respecto de...:
        %Nodo central (ii):
        dfI_du_C = (uv(ii+1)-uv(ii-1))/(2*hx) + 2*epsilon/hx^2;
        %Nodo izquierdo (ii-1)
        dfI_du_L = -uv(ii)/(2*hx) - epsilon/hx^2;
        %Nodo derecho (ii+1):
        dfI_du_R = uv(ii)/(2*hx) - epsilon/hx^2;

        %Almacenamos derivadas:
        filas = cat(1, filas, ii, ii, ii);
        cols  = cat(1, cols, ii, ii-1, ii+1);
        vals  = cat(1, vals, dfI_du_C, dfI_du_L, dfI_du_R);

    end

end

%Frontera izquierda. Imponemos  $u_1 = 1$ :
f(1) = uv(1) - 1.0;
if CalcJ
    filas = cat(1, filas, 1);
    cols  = cat(1, cols, 1);
    vals  = cat(1, vals, 1);
end

%Frontera derecha. Imponemos  $u_{Nx} = 0$ :
f(Nx) = uv(Nx) - 0.0;
if CalcJ

```

```
        filas    = cat(1, filas, Nx);
        cols     = cat(1, cols, Nx);
        vals     = cat(1, vals, 1);
    end

    %Construimos jacobiano:
    if CalcJ
        J        = sparse(filas, cols, vals, Nx, Nx);
    else
        J        = NaN;
    end

end
```

Published with MATLAB® R2025a

```

function [f,J] = Burgers1D_DF_Residuo2(xv, epsilon, uv, CalcJ)

%Distancia entre nodos (OJO! Se supone mallado uniforme):
Nx      = length(xv);
hx      = xv(2)-xv(1);

%Reservamos memoria para residuo y Jacobiano:
f       = zeros(Nx,1);
if CalcJ
    filas = [];
    cols  = [];
    vals  = [];
end

%Puntos interiores:
for ii=2:Nx-1

    %Imponemos
    %  $u_i(u_i - u_{i-1})/hx - \dots$ 
    %  $\epsilon(u_{i+1} - 2u_i + u_{i-1})/hx^2 = 0$ 
    f(ii) = uv(ii) * (uv(ii) - uv(ii-1))/hx - ...
            epsilon*( uv(ii+1)-2*uv(ii)+uv(ii-1) )/hx^2;
    if CalcJ

        %Derivadas de f(I) respecto de...:
        %Nodo central (ii):
        dfI_du_C = (2*uv(ii)-uv(ii-1))/hx + 2*epsilon/hx^2;
        %Nodo izquierdo (ii-1)
        dfI_du_L = -uv(ii)/hx - epsilon/hx^2;
        %Nodo derecho (ii+1):
        dfI_du_R = - epsilon/hx^2;

        %Almacenamos derivadas:
        filas = cat(1, filas, ii, ii, ii);
        cols  = cat(1, cols, ii, ii-1, ii+1);
        vals  = cat(1, vals, dfI_du_C, dfI_du_L, dfI_du_R);

    end

end

%Frontera izquierda. Imponemos  $u_1 = 1$ :
f(1) = uv(1) - 1.0;
if CalcJ
    filas = cat(1, filas, 1);
    cols  = cat(1, cols, 1);
    vals  = cat(1, vals, 1);
end

%Frontera derecha. Imponemos  $u_{Nx} = 0$ :
f(Nx) = uv(Nx) - 0.0;
if CalcJ

```

```
        filas    = cat(1, filas, Nx);
        cols     = cat(1, cols, Nx);
        vals     = cat(1, vals, 1);
    end

    %Construimos jacobiano:
    if CalcJ
        J        = sparse(filas, cols, vals, Nx, Nx);
    else
        J        = NaN;
    end

end
```

Published with MATLAB® R2025a